

INTRODUZIONE ALL'HACKING DI FABIO RENNA

Questo documento introduce i principi fondamentali dell'Ethical Hacking e della Cybersecurity, illustrando gli strumenti e le metodologie utilizzate per analizzare, testare e progettare i sistemi informatici.

Attraverso esercitazioni pratiche in ambiente virtuale, vengono approfonditi i concetti di rete, DNS, comunicazioni HTTP/HTTPS e tecniche di analisi del traffico con Wireshark.

PROGETTO
EPICODE N°1

Nome: Fabio Renna

Data: 16/10/2025

Oggetto: Introduzione all'Hacking di Fabio Renna

Introduzione:

Questo progetto ha come obiettivo la simulazione di un'architettura client-server e l'analisi delle differenze tra traffico **HTTP** e **HTTPS** con il tool **Wireshark**.

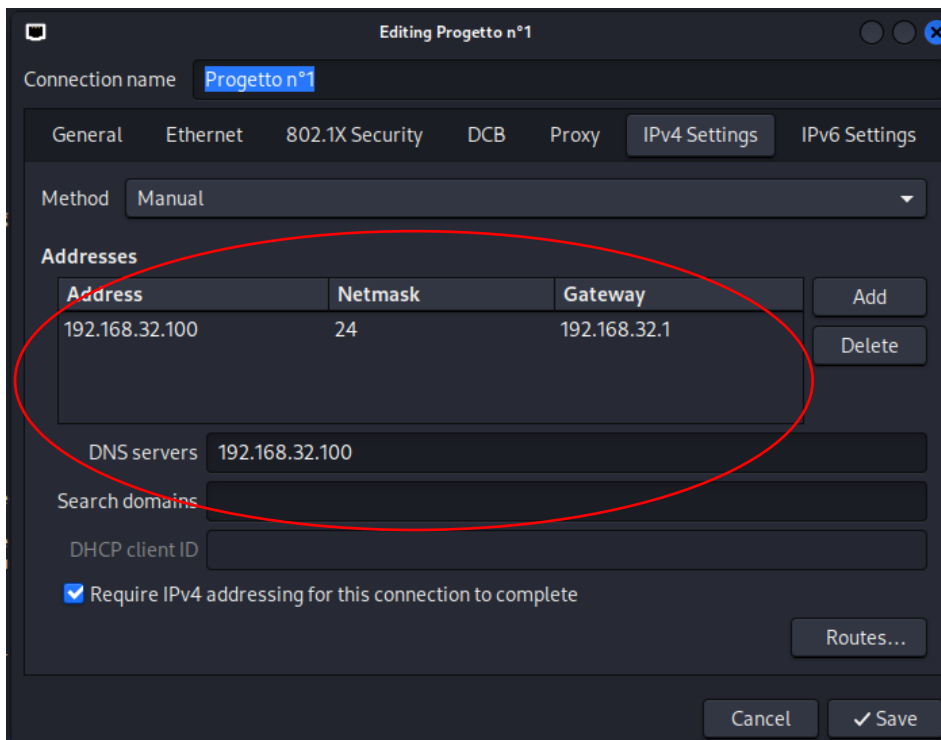
L'esperimento viene eseguito in ambiente virtuale, utilizzando due macchine collegate in rete interna:

- Kali Linux (server) con IP **192.168.32.100**
- Windows 10 pro (client) con IP **192.168.32.101**

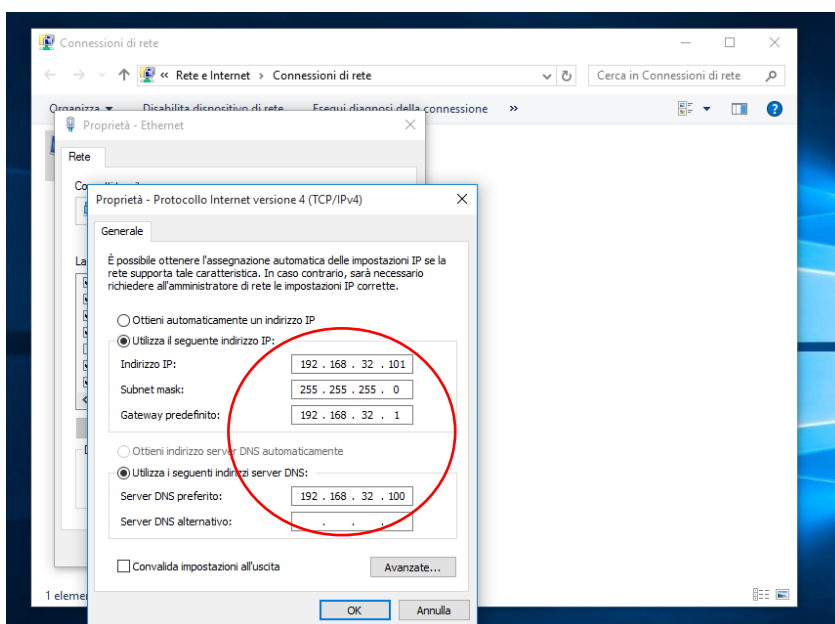
Per rendere l'ambiente più "realistico", ho scelto di utilizzare **Inetsim**, che simula automaticamente diversi servizi di rete, tra cui **HTTP**, **HTTPS** e **DNS**.

Svolgimento:

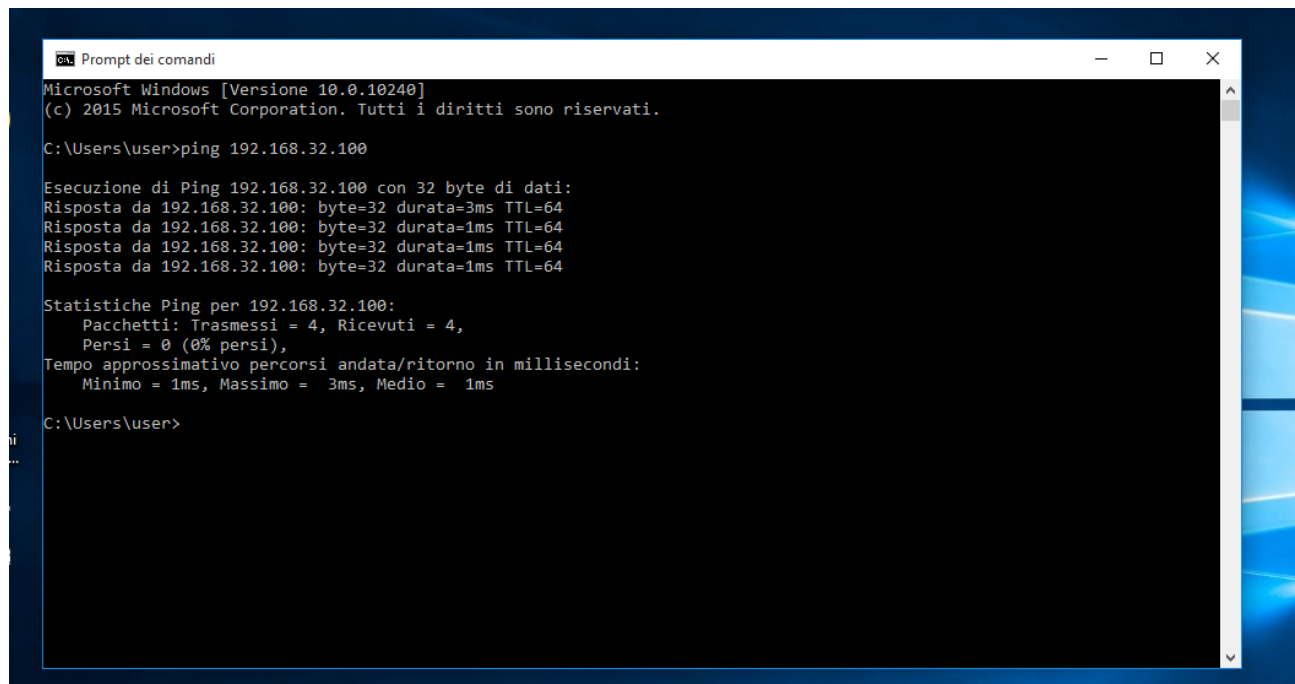
Come primo passo abbiamo modificato le impostazioni di rete di ciascuna macchina, partendo da **Kali Linux**. Abbiamo creato una nuova connessione Ethernet rinominandola "Progetto n°1" con configurazione manuale, impostando l'indirizzo IP **192.168.32.100**, il **Default Gateway** (non strettamente necessario, poiché la rete è interna, impostato su **192.168.32.1**) e il **DNS** server anch'esso su **192.168.32.100**.



Successivamente abbiamo modificato anche le impostazioni di rete su Windows, accedendo alle proprietà del **Protocollo Internet versione 4 (TCP/IPv4)**. Abbiamo impostato l'indirizzo IP su **192.168.32.101**, il **Default Gateway** (non strettamente necessario in rete interna) su **192.168.32.1**, e il **DNS** server, uguale a quello di Kali, su **192.168.32.100**.



Infine, per verificare che le due macchine fossero correttamente collegate alla stessa rete, abbiamo effettuato un ping da Windows verso Kali, ottenendo risposta positiva.



```
Prompt dei comandi
Microsoft Windows [Versione 10.0.10240]
(c) 2015 Microsoft Corporation. Tutti i diritti sono riservati.

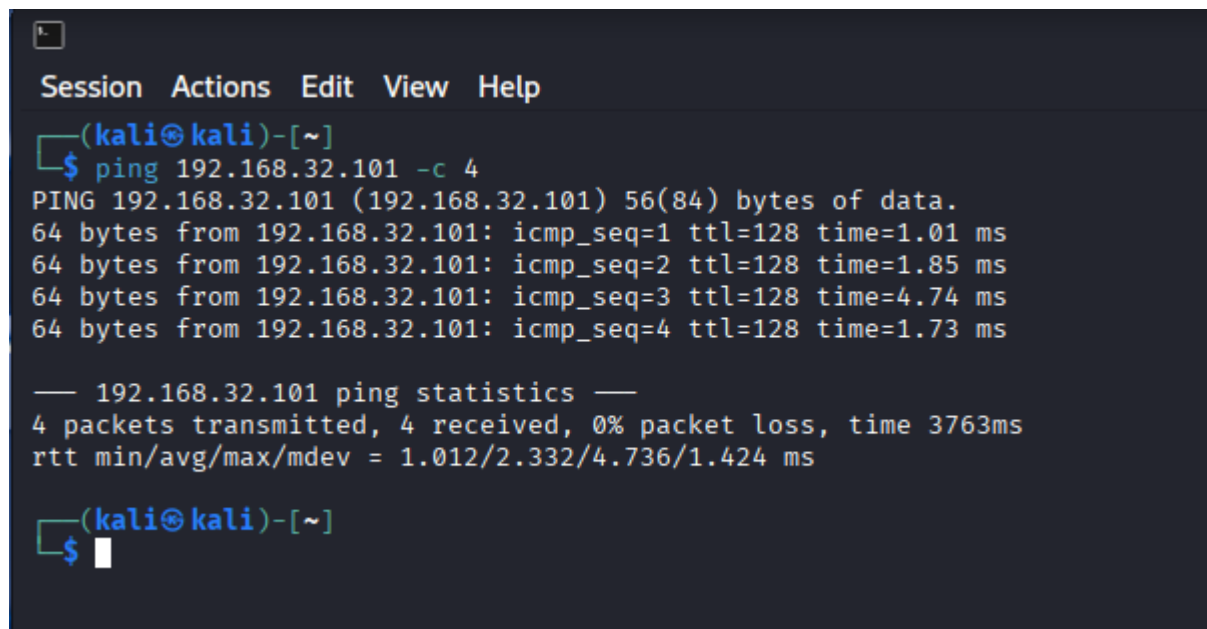
C:\Users\user>ping 192.168.32.100

Esecuzione di Ping 192.168.32.100 con 32 byte di dati:
Risposta da 192.168.32.100: byte=32 durata=3ms TTL=64
Risposta da 192.168.32.100: byte=32 durata=1ms TTL=64
Risposta da 192.168.32.100: byte=32 durata=1ms TTL=64
Risposta da 192.168.32.100: byte=32 durata=1ms TTL=64

Statistiche Ping per 192.168.32.100:
    Pacchetti: Trasmessi = 4, Ricevuti = 4,
    Persi = 0 (0% persi),
    Tempo approssimativo percorsi andata/ritorno in millisecondi:
        Minimo = 1ms, Massimo = 3ms, Medio = 1ms

C:\Users\user>
```

Successivamente abbiamo eseguito un ulteriore ping da Kali verso Windows per confermare definitivamente la corretta configurazione della rete.

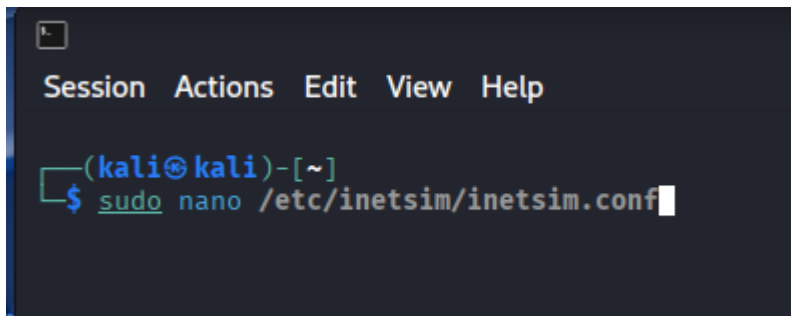


```
Session Actions Edit View Help
(kali㉿kali)-[~]
$ ping 192.168.32.101 -c 4
PING 192.168.32.101 (192.168.32.101) 56(84) bytes of data.
64 bytes from 192.168.32.101: icmp_seq=1 ttl=128 time=1.01 ms
64 bytes from 192.168.32.101: icmp_seq=2 ttl=128 time=1.85 ms
64 bytes from 192.168.32.101: icmp_seq=3 ttl=128 time=4.74 ms
64 bytes from 192.168.32.101: icmp_seq=4 ttl=128 time=1.73 ms

— 192.168.32.101 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3763ms
rtt min/avg/max/mdev = 1.012/2.332/4.736/1.424 ms

(kali㉿kali)-[~]
$
```

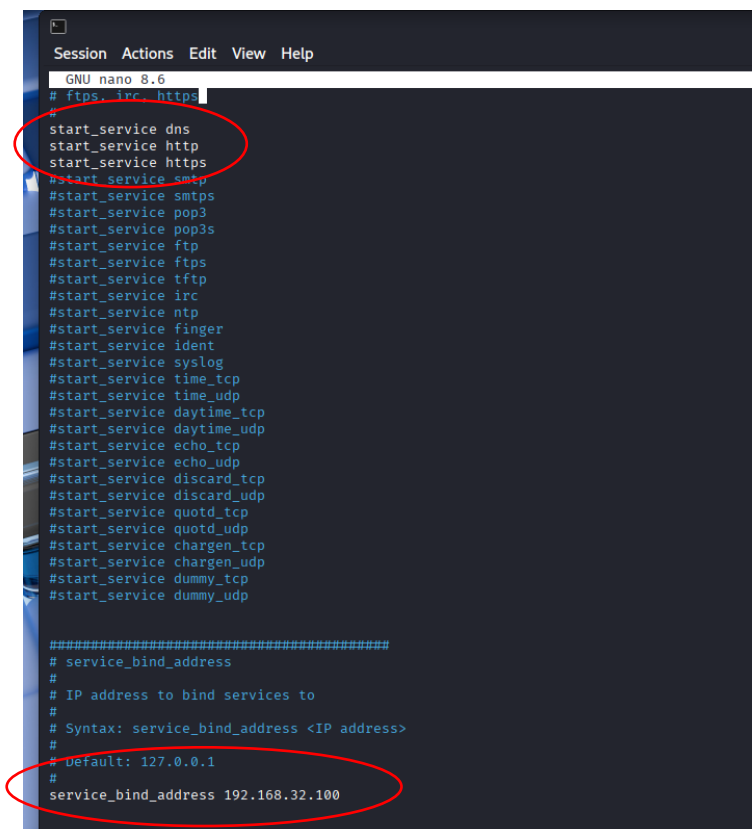
Ora non ci resta che attivare e configurare i servizi **HTTP**, **HTTPS** e **DNS** tramite Inetsim su Kali, aprendo il file di configurazione con il comando:



```
Session Actions Edit View Help

(kali㉿kali)-[~]
$ sudo nano /etc/inetsim/inetsim.conf
```

Dalla schermata successiva abbiamo disabilitato il commento, rimuovendo il simbolo “#” dalle seguenti stringhe: **start_service dns**, **start_service http** e **start_service https** per determinare l’attivazione dei servizi all’avvio di Inetsim e aggiungendo alla voce “**service_bind_address**” l’indirizzo **192.168.32.100**.



```
Session Actions Edit View Help

GNU nano 8.6
# ftps.irc.https
#
start_service dns
start_service http
start_service https
#start_service smtp
#start_service smtps
#start_service pop3
#start_service pop3s
#start_service ftp
#start_service ftps
#start_service tftp
#start_service irc
#start_service ntp
#start_service finger
#start_service ident
#start_service syslog
#start_service time_tcp
#start_service time_udp
#start_service daytime_tcp
#start_service daytime_udp
#start_service echo_tcp
#start_service echo_udp
#start_service discard_tcp
#start_service discard_udp
#start_service quotd_tcp
#start_service quotd_udp
#start_service chargen_tcp
#start_service chargen_udp
#start_service dummy_tcp
#start_service dummy_udp

#####
# service_bind_address
#
# IP address to bind services to
#
# Syntax: service_bind_address <IP address>
#
# default: 127.0.0.1
#
service_bind_address 192.168.32.100
```

Una volta salvato, abbiamo avviato Inetsim e ci siamo messi in ascolto sulla rete.

```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ sudo inetsim  
[sudo] password for kali:  
INetSim 1.3.2 (2020-05-19) by Matthias Eckert & Thomas Hungenberg  
Using log directory: /var/log/inetsim/  
Using data directory: /var/lib/inetsim/  
Using report directory: /var/log/inetsim/report/  
Using configuration file: /etc/inetsim/inetsim.conf  
Parsing configuration file.  
Configuration file parsed successfully.  
INetSim main process started (PID 2708)  
Session ID: 2708  
Listening on: 192.168.32.100  
Real Date/Time: 2025-10-17 04:56:30  
Fake Date/Time: 2025-10-17 04:56:30 (Delta: 0 seconds)  
Forking services ...  
* dns_53_tcp_udp - started (PID 2710)  
Can't locate object method "main_loop" via package "Net::DNS::Nameserver" at /usr/share/perl5/INetSim/DNS.pm line 69.  
* https_443_tcp - started (PID 2712)  
* http_80_tcp - started (PID 2711)  
done.  
Simulation running.
```

Durante le prove ho riscontrato che il servizio DNS di Inetsim non si avviava correttamente, restituendo l'errore:

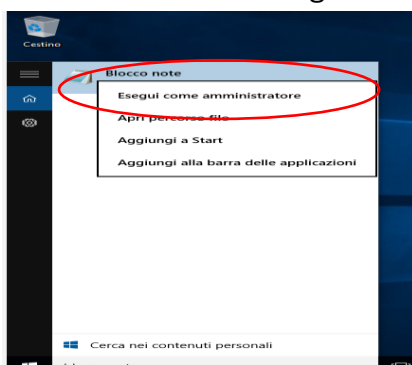
"Can't locate object method 'main_loop' via package 'Net::DNS::Nameserver'".

Tale anomalia è dovuta a una incompatibilità tra la versione 1.3.2 di Inetsim e il modulo **Perl Net::DNS** presente nelle release più recenti di Kali Linux. Ho effettuato una ricerca sulla problematica e trovato segnalazioni analoghe in forum tecnici (***ad es. discussioni su Reddit -***

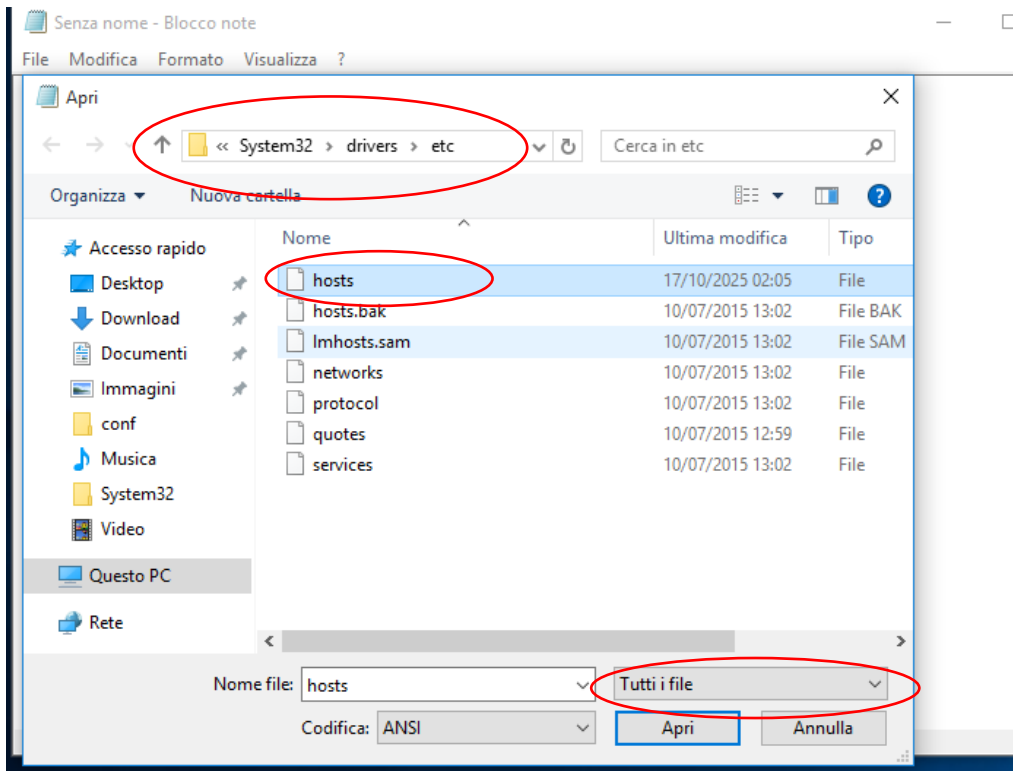
"Net::DNS module has been rewritten and doesn't work properly with Inetsim on the latest")

Per ovviare al problema ho adottato le seguenti strategie pratiche:

1. Ho disattivato il servizio DNS di Inetsim (mantenendo attivi i servizi HTTP e HTTPS) nel file di configurazione **/etc/inetsim/inetsim.conf**.
2. Ho simulato la risoluzione del dominio **epicode.internal** sul client Windows aggiungendo manualmente la mappatura **192.168.32.100** epicode.internal nel file **hosts**. Per farlo ho eseguito il Blocco Note con **privilegi di amministratore**.



aperto **C:\Windows\System32\drivers\etc\hosts** (impostando in “Tutti i file” la visualizzazione), e inserito la riga in fondo al file.

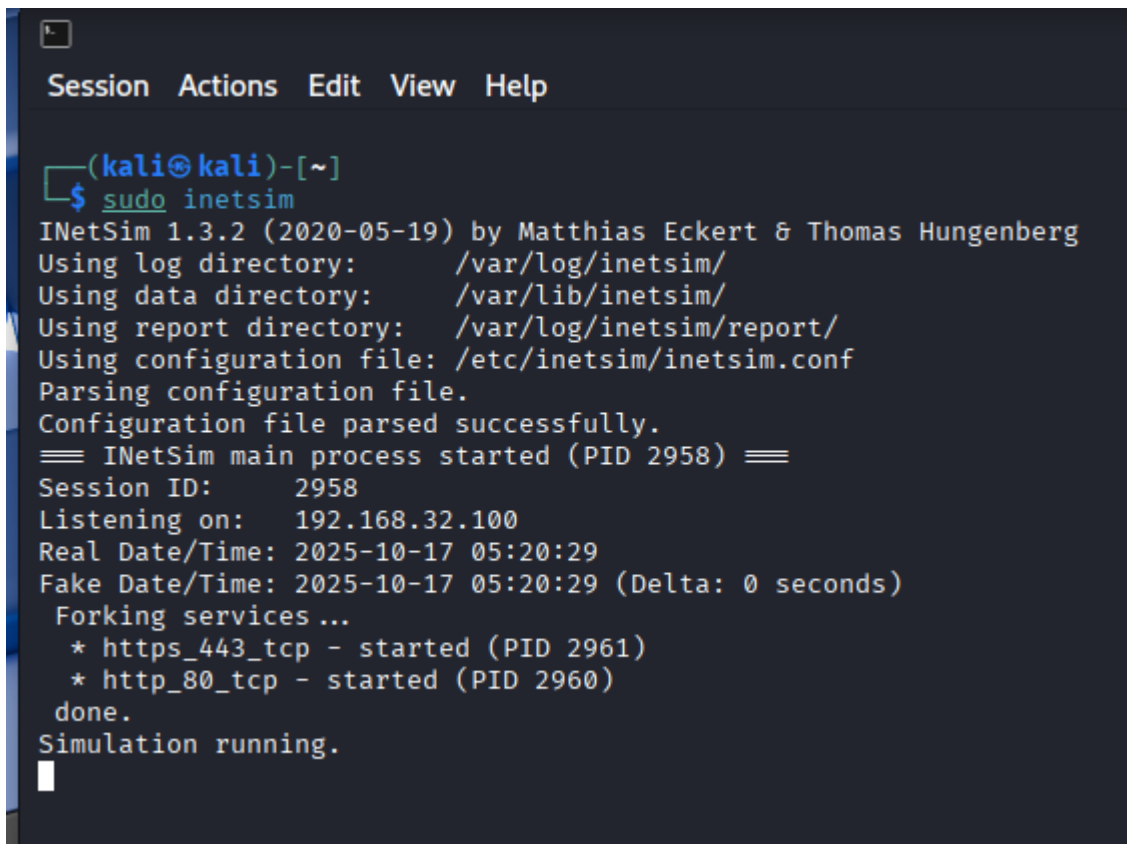


The screenshot shows a Notepad window titled "hosts - Blocco note" displaying the content of the hosts file. The text is as follows:

```
File Modifica Formato Visualizza ?
# Copyright (c) 1993-2009 Microsoft Corp.
#
# This is a sample HOSTS file used by Microsoft TCP/IP for Windows.
#
# This file contains the mappings of IP addresses to host names. Each
# entry should be kept on an individual line. The IP address should
# be placed in the first column followed by the corresponding host name.
# The IP address and the host name should be separated by at least one
# space.
#
# Additionally, comments (such as these) may be inserted on individual
# lines or following the machine name denoted by a '#' symbol.
#
# For example:
#
# 102.54.94.97 rhino.acme.com      # source server
# 38.25.63.10  x.acme.com        # x client host
#
# localhost_name resolution is handled within DNS itself.
# 127.0.0.1    localhost
# ::1          localhost
192.168.32.100 epicode.internal
```

A red circle is drawn around the last three lines of the file, which are the entries for localhost and the custom entry for epicode.internal.

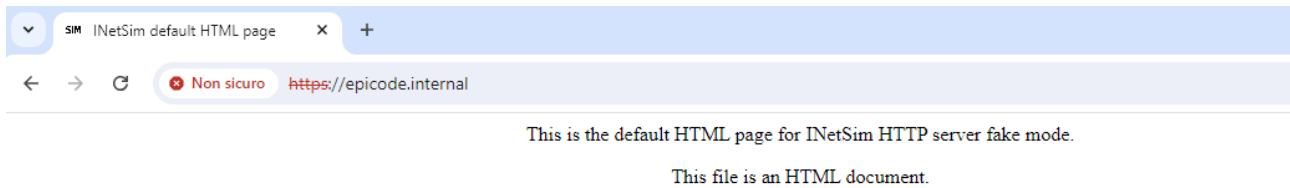
Ora che abbiamo tutte le configurazioni pronte, siamo andati ad avviare **Inetsim** con i soli servizi **HTTP** e **HTTPS** attivi e a verificare la raggiungibilità di <http://epicode.internal> e <https://epicode.internal> dal browser della VM Windows.

A terminal window with a dark background and light-colored text. The window has a title bar with a close button and a menu bar with 'Session', 'Actions', 'Edit', 'View', and 'Help'. The prompt is '(kali㉿kali)-[~]'. The user enters '\$ sudo inetsim'. The output shows 'INetSim 1.3.2 (2020-05-19) by Matthias Eckert & Thomas Hungenberg' followed by log, data, and report directories, and the configuration file path. It then shows 'Parsing configuration file.' and 'Configuration file parsed successfully.' followed by a separator line '=== INetSim main process started (PID 2958) ==='. Below this, it shows 'Session ID: 2958', 'Listening on: 192.168.32.100', 'Real Date/Time: 2025-10-17 05:20:29', and 'Fake Date/Time: 2025-10-17 05:20:29 (Delta: 0 seconds)'. Then it says 'Forking services ...' followed by '* https_443_tcp - started (PID 2961)' and '* http_80_tcp - started (PID 2960)', then 'done.' and 'Simulation running.' with a cursor at the end.

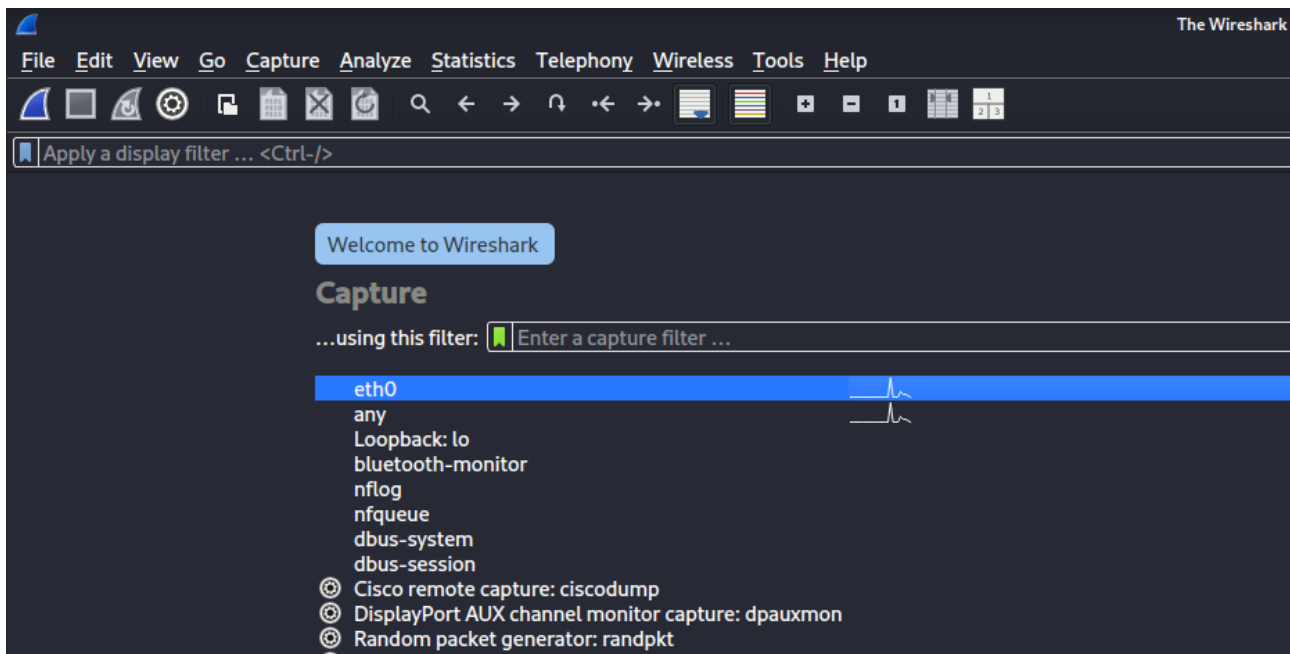
```
(kali㉿kali)-[~]
$ sudo inetsim
INetSim 1.3.2 (2020-05-19) by Matthias Eckert & Thomas Hungenberg
Using log directory:      /var/log/inetsim/
Using data directory:     /var/lib/inetsim/
Using report directory:   /var/log/inetsim/report/
Using configuration file: /etc/inetsim/inetsim.conf
Parsing configuration file.
Configuration file parsed successfully.
=== INetSim main process started (PID 2958) ===
Session ID:      2958
Listening on:    192.168.32.100
Real Date/Time:  2025-10-17 05:20:29
Fake Date/Time: 2025-10-17 05:20:29 (Delta: 0 seconds)
Forking services ...
 * https_443_tcp - started (PID 2961)
 * http_80_tcp  - started (PID 2960)
done.
Simulation running.
█
```

Per la visualizzazione della pagina web in **HTTP** basta inserire nell'url <http://epicode.internal> e Windows risolve il nome utilizzando il file hosts (nel caso il servizio DNS di inetsim non sia disponibile) e lo mappa all'indirizzo **192.168.32.100**, che ospita i servizi **HTTP** e **HTTPS** forniti da inetsim su Kali. In questo modo il browser si collega al server fittizio e visualizza la pagina simulata.

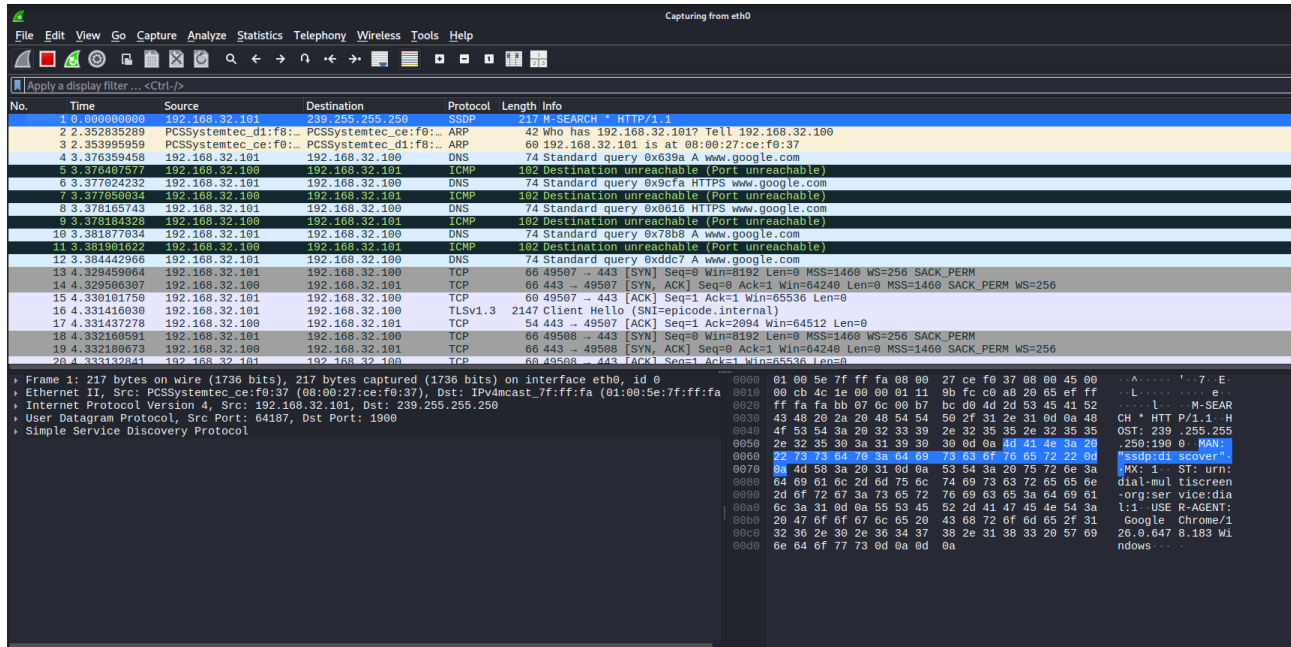
Per l'accesso **HTTPS** è stato mostrato un avviso di certificato non attendibile: è normale in ambiente di laboratorio e, accettando il rischio, si può proseguire per analizzare il traffico cifrato.



Come strumento di analisi traffico, abbiamo utilizzato il tool pre-installato su Kali: **Wireshark**. Mentre Inetsim è in esecuzione, siamo andati ad eseguire il programma impostando su **eth0**.

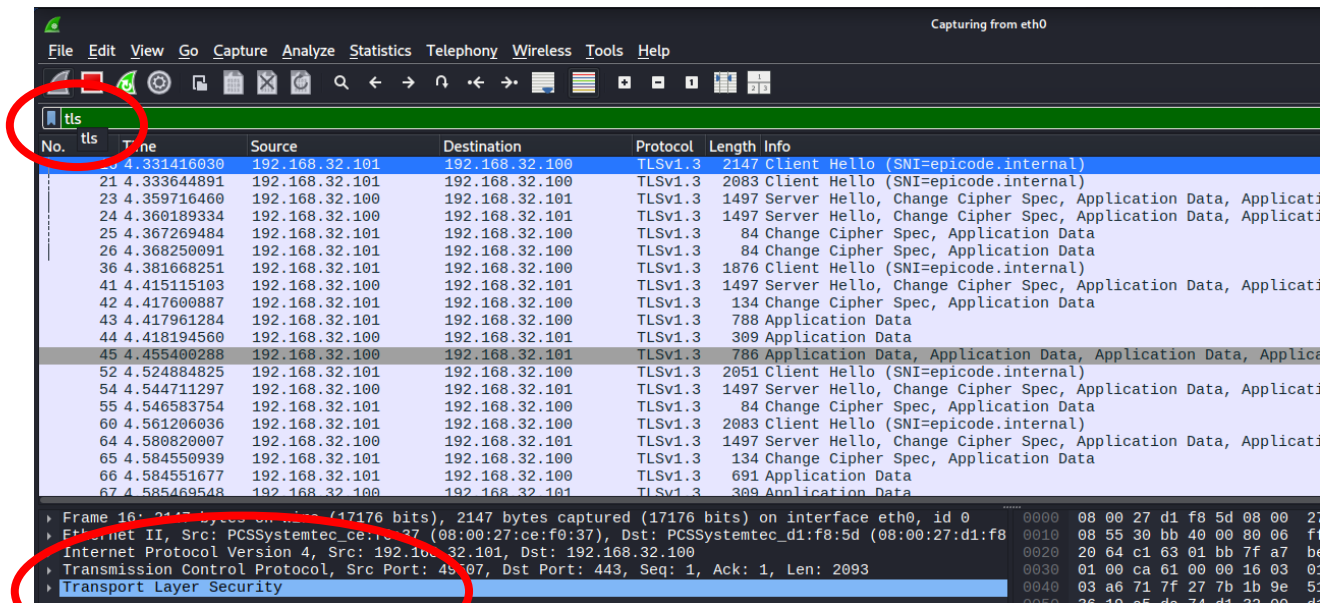


Davanti a noi è comparsa la schermata con tutti i pacchetti inviati e ricevuti tra i due host (Windows e Kali), inclusi quelli DNS.



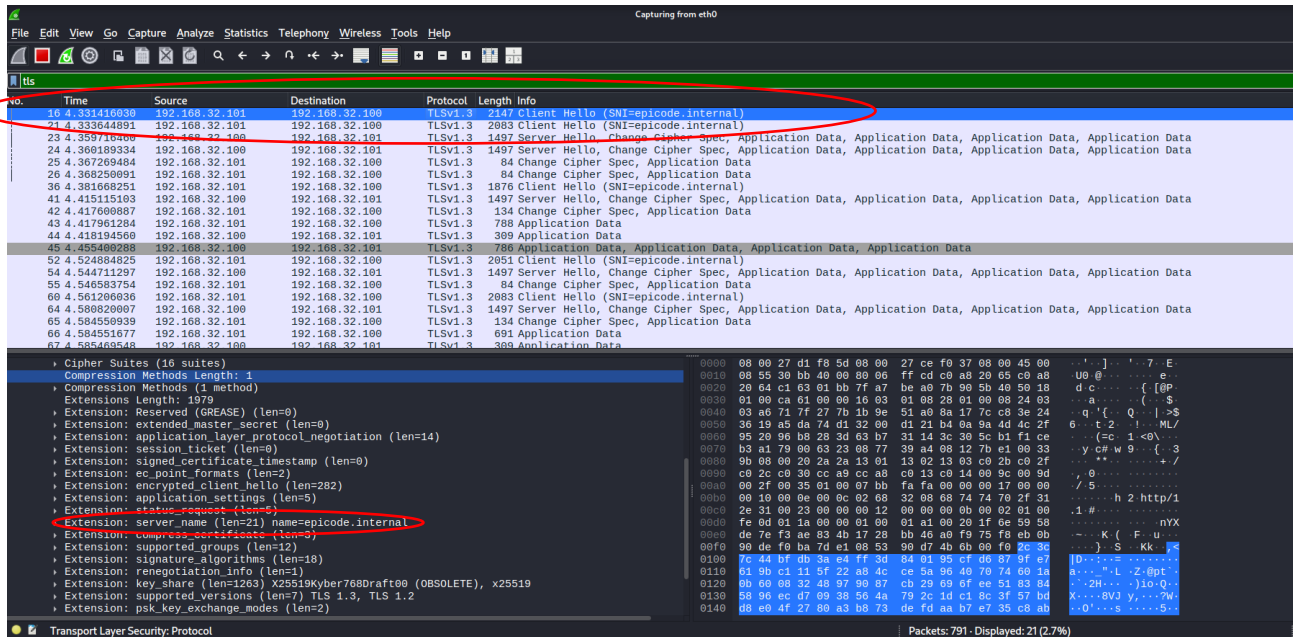
Per rendere l'analisi più chiara e concentrarci esclusivamente sul traffico **HTTPS**, abbiamo applicato il filtro `tls` (protocollo per **HTTPS**) in **Wireshark**.

In questo modo vengono visualizzati soltanto i pacchetti appartenenti al protocollo **TLS**, cioè le comunicazioni cifrate che transitano sulla porta **443**.

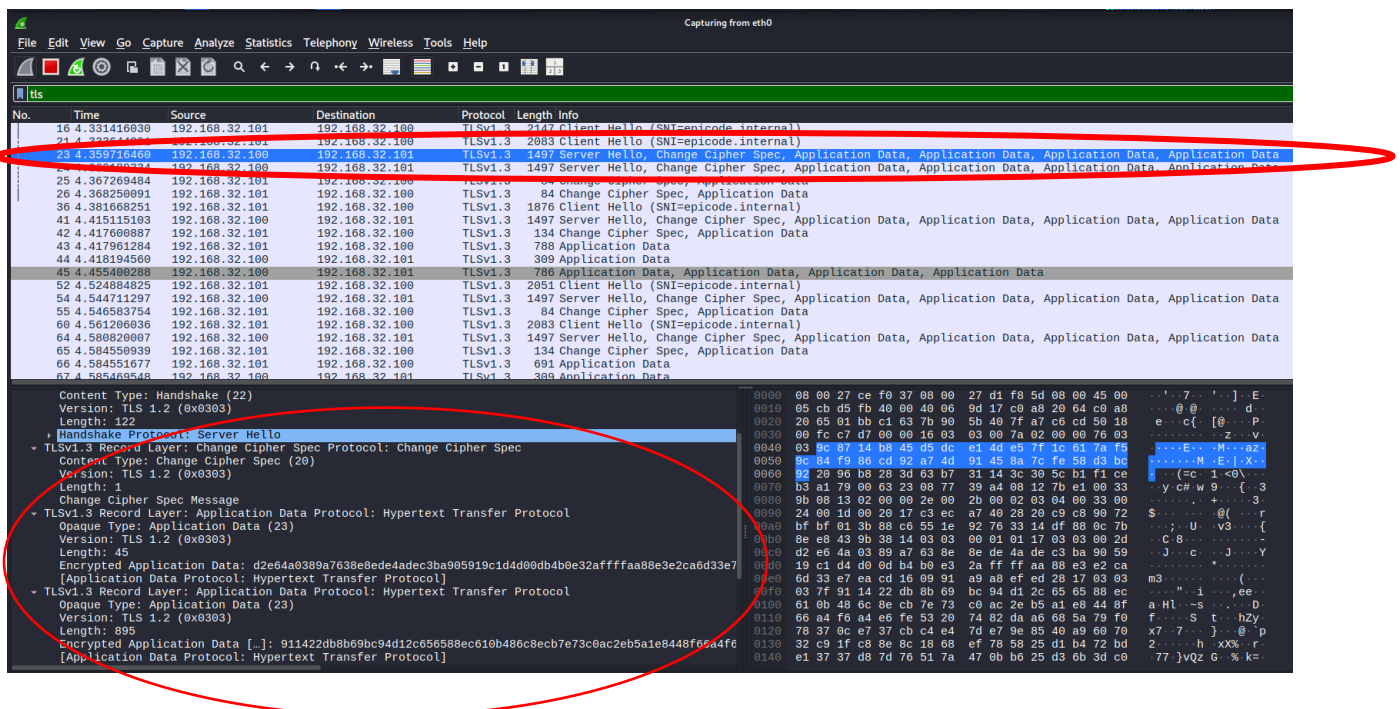


Dopo aver applicato il filtro `tls`, sono stati mostrati i pacchetti relativi al protocollo di sicurezza **TLS**, che costituisce la base delle comunicazioni **HTTPS**.

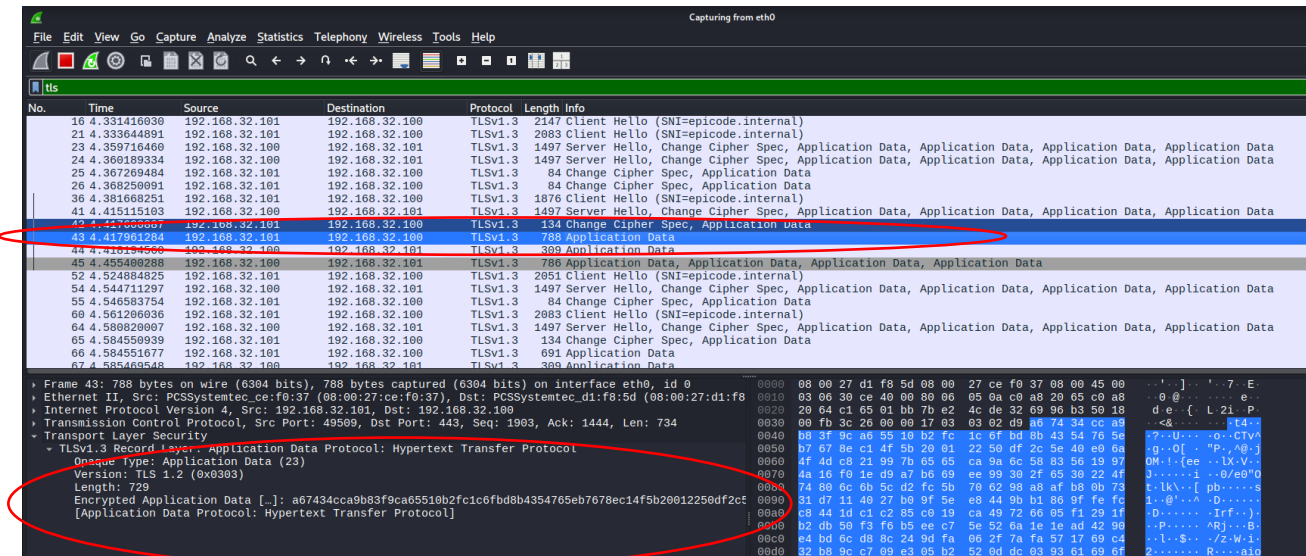
Analizzando il pacchetto **Client Hello**, si osserva che il client (**192.168.32.101**) inizia la negoziazione **TLS** con il server (192.168.32.100), specificando il dominio **epicode.internal** come **SNI**.



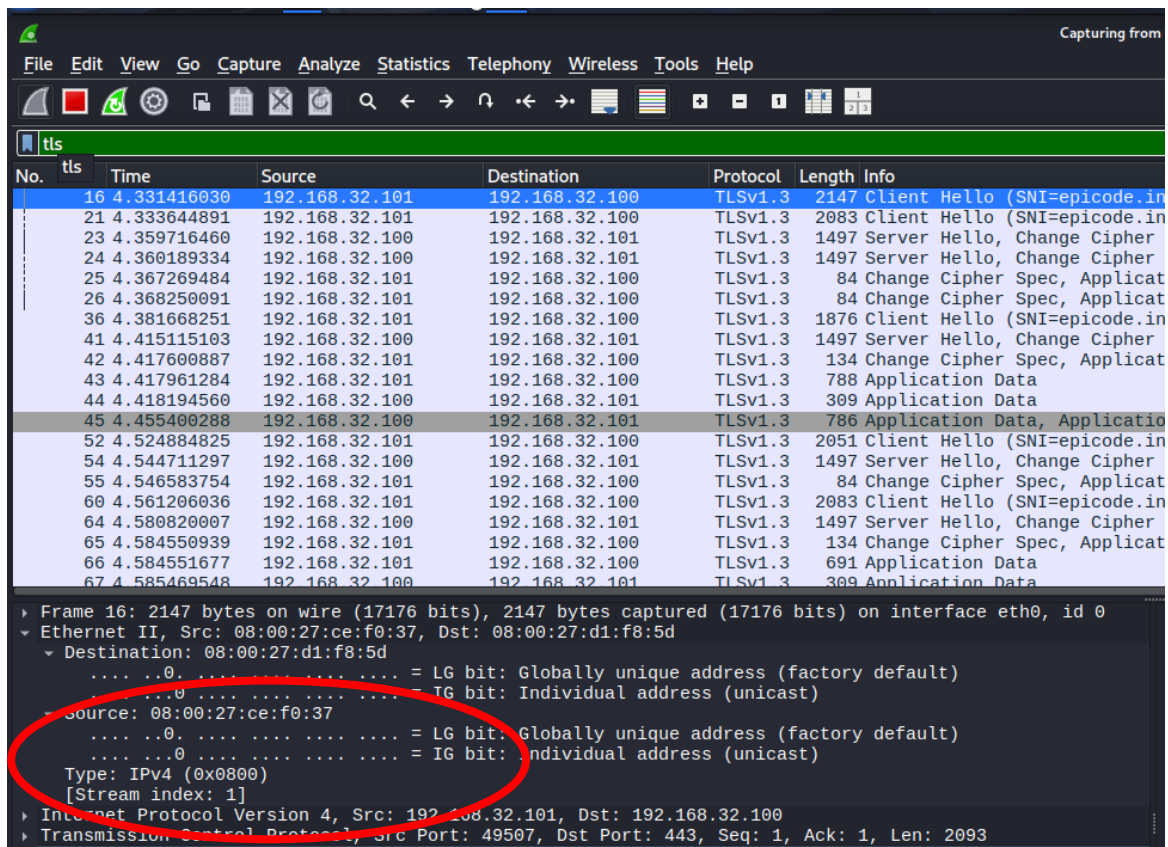
Il server ha risposto con un **Server Hello**, accettando una delle cipher suite proposte e confermando l'uso del protocollo **TLS 1.3. (tls 1.2 0x0303)**



Da questo punto in avanti, tutto il traffico è risultato cifrato: **Wireshark** ha mostra i pacchetti come **“Encrypted Application Data”**, a conferma della protezione dei dati scambiati.

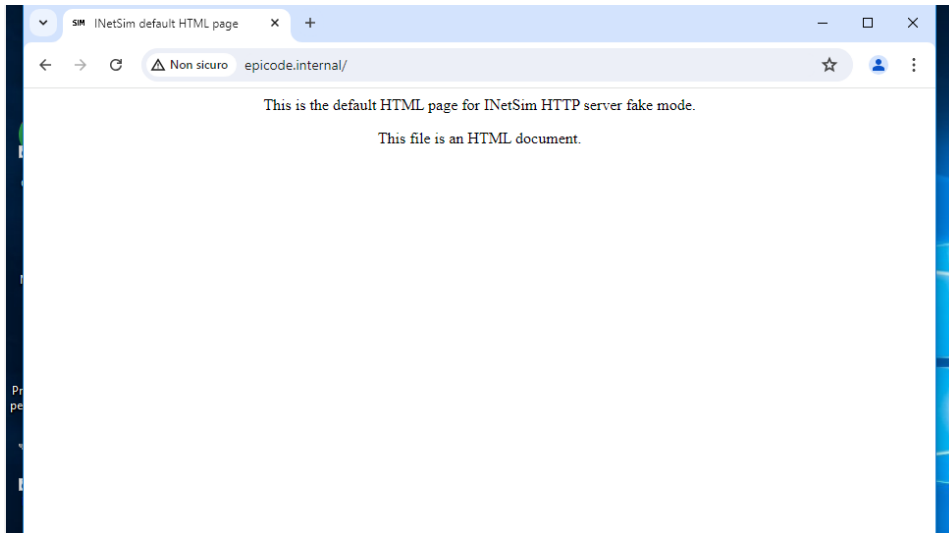


All'interno dei pacchetti è stato possibile anche notare il MAC Address della macchina sorgente Windows (source: **08:00:27:ce:f0:37**) e il MAC Address della macchina destinataria Kali (Destination: **08:00:27:d1:f8:5d**)



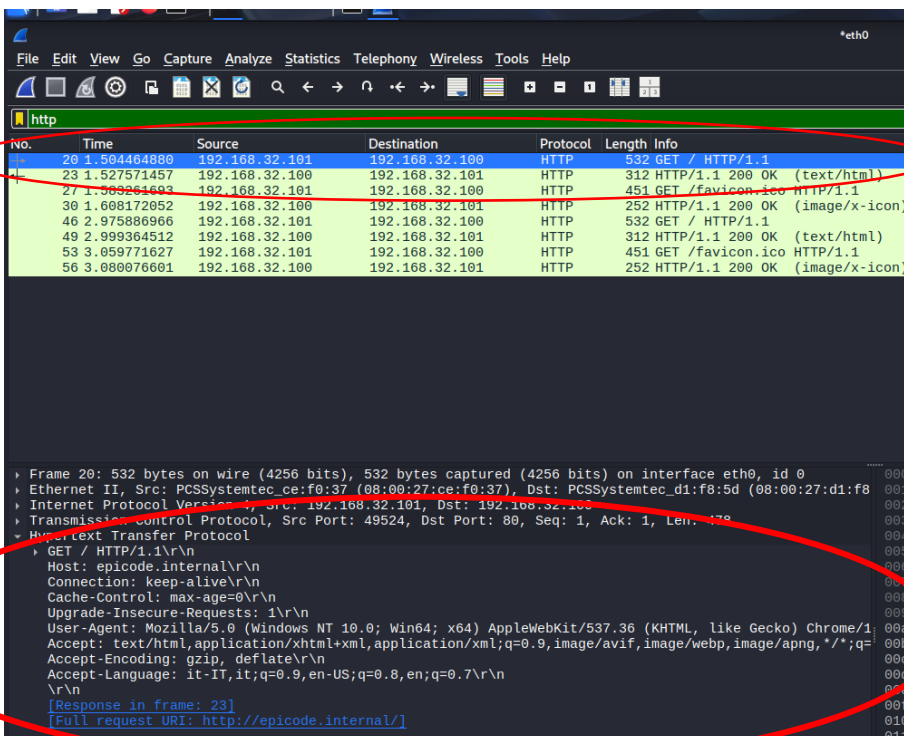
Dopo aver completato l'analisi del traffico cifrato **HTTPS**, abbiamo ripetuto la stessa procedura concentrandoci questa volta sul protocollo **HTTP**, per osservare in modo diretto le differenze tra una comunicazione in chiaro e una protetta da **TLS**.

Dal client Windows abbiamo poi aperto il browser e inserito l'indirizzo configurato nel file hosts, in questo caso: <http://epicode.internal>



Durante la cattura, nella schermata di **Wireshark** sono apparsi i pacchetti di tipo **HTTP**, con richieste e risposte completamente leggibili.

Nel dettaglio di un pacchetto, è stato possibile osservare l'intera richiesta del client, come nel seguente esempio:



Il server, successivamente, ha risposto con un messaggio **HTTP/1.1 200 OK**, contenente l'header, il MAC Address sorgente (in questo caso Kali - Source: **08:00:27:d1:f8:5d**), il MAC Address destinazione (in questo caso Windows – Destination: **08:00:27:ce:f0:37**) con il corpo della pagina in chiaro.

The image shows a Wireshark packet capture of an HTTP response. The packet list at the top shows several frames, with frame 23 selected. The packet details pane for frame 23 shows the following information:

- Frame 23: 312 bytes on wire (2496 bits), 312 bytes captured (2496 bits) on interface eth0, id 0
- Ethernet II, Src: PCSSystemtec_d1:f8:5d (08:00:27:d1:f8:5d), Dst: PCSSystemtec_ce:f0:37 (08:00:27:ce:f0:37)
- Destination: 08:00:27:ce:f0:37
- Source: 08:00:27:d1:f8:5d
- Type: IPv4 (0x0800)
- [Stream index: 0]
- Internet Protocol Version 4, Src: 192.168.32.100, Dst: 192.168.32.101
- Transmission Control Protocol, Src Port: 80, Dst Port: 49524, Seq: 151, Ack: 479, Len: 258
- [2 Reassembled TCP Segments (408 bytes): #22(150), #23(258)]
- Hypertext Transfer Protocol
- HTTP/1.1 200 OK\r\n
- Content-Length: 258\r\n
- Server: INetSim HTTP Server\r\n
- Connection: Close\r\n
- Date: Fri, 17 Oct 2025 16:20:35 GMT\r\n
- Content-Type: text/html\r\n
- \r\n
- [Request in frame: 20]
- [Time since request: 0.023106577 seconds]
- [Request URI: /]
- [Full request URI: http://epicode.internal/]
- File Data: 258 bytes
- Line-based text data: text/html (10 lines)
- <html>\n
- <head>\n
- <title>INetSim default HTML page</title>\n
- </head>\n
- <body>\n
- <p>\n
- <p align="center">This is the default HTML page for INetSim HTTP server fake mode.</p>\n
- <p align="center">This file is an HTML document.</p>\n
- </body>\n
- </html>\n

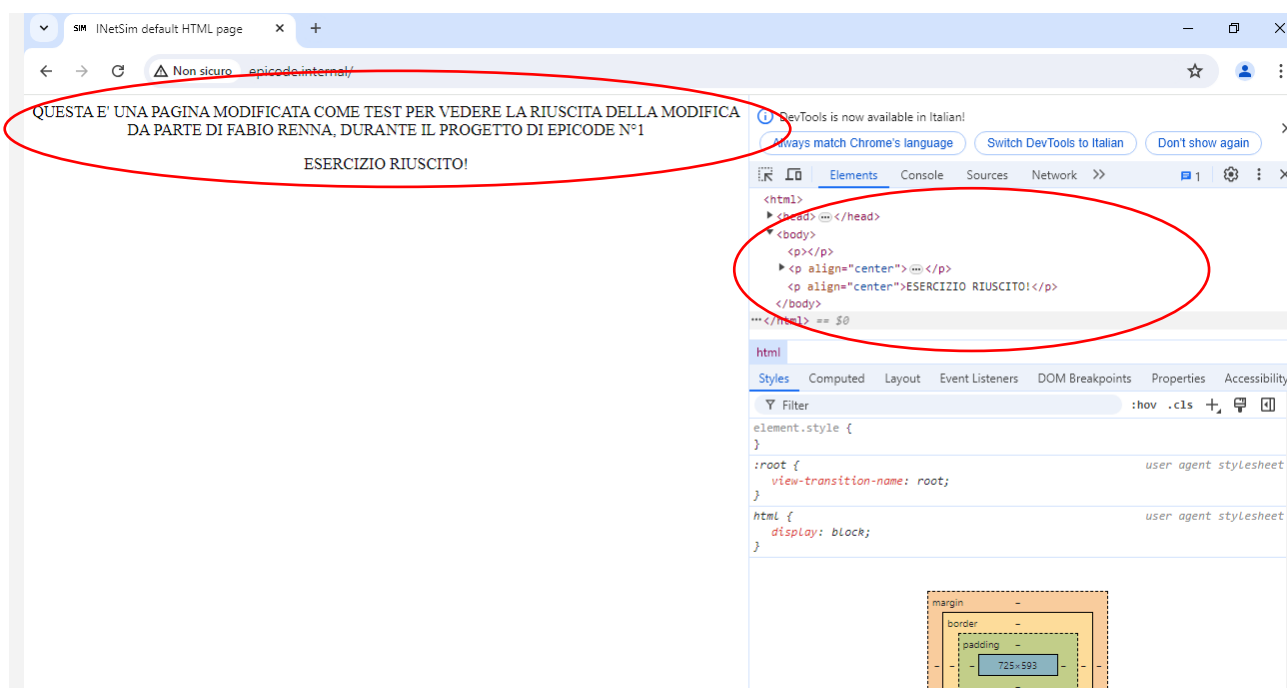
Questo dimostra che, nel protocollo **HTTP**, l'intero contenuto del messaggio (incluse intestazioni e dati trasmessi) è accessibile e ispezionabile senza alcuna protezione crittografica.

Bonus:

Come test aggiuntivo, ho voluto verificare la possibilità di modificare il contenuto della pagina **HTML** servita da **Inetsim**, agendo direttamente dal browser del client Windows.

Una volta aperta la pagina <http://epicode.internal>, ho utilizzato lo strumento Ispeziona elemento per analizzare il codice sorgente della pagina web restituita dal server.

Poiché il protocollo **HTTP** trasmette i dati in chiaro e il contenuto della pagina è in semplice formato **HTML**, è possibile visualizzare e modificare dinamicamente il testo lato client senza alcuna restrizione di sicurezza.



La modifica è stata eseguita solo lato client, quindi non ha alterato il file **HTML** originale sul server, ma dimostra come sia possibile intervenire sui contenuti web quando il traffico non è cifrato e la pagina è accessibile in chiaro.

Questo esperimento sottolinea ulteriormente l'importanza della cifratura **HTTPS**, che impedisce la manipolazione o l'intercettazione dei dati durante la trasmissione.

Conclusione:

L'intero esercizio ha permesso di comprendere in modo pratico le differenze fondamentali tra i protocolli **HTTP** e **HTTPS**, evidenziando il ruolo della cifratura TLS nella protezione delle comunicazioni.

Attraverso l'utilizzo combinato di **Inetsim** e **Wireshark**, sono stati catturati e analizzati i pacchetti di rete ai diversi livelli del modello ISO/OSI:

- **Livello Ethernet:** identificazione e associazione dei MAC address tra client e server;
- **Livello IP:** instradamento del traffico e comunicazione diretta tra gli host;
- **Livello di trasporto:** funzionamento del protocollo **TCP** e delle porte 80 (**HTTP**) e 443 (**HTTPS**);
- **Livello applicativo:** confronto tra il traffico **HTTP**, visibile in chiaro, e quello **HTTPS**, cifrato tramite **TLS**.

L'analisi ha evidenziato che, nel caso **HTTP**, è possibile visualizzare in chiaro ogni dettaglio della comunicazione (richieste, header e contenuti), mentre nel caso **HTTPS** tutto il traffico successivo all'handshake **TLS** risulta cifrato e quindi illeggibile, garantendo **riservatezza**, **integrità** e **autenticità** dei dati trasmessi.

In conclusione, il laboratorio ha raggiunto pienamente l'obiettivo:

ho potuto osservare e comprendere in modo concreto il funzionamento dei protocolli di rete, la struttura dei pacchetti e la protezione offerta da **TLS**, con una dimostrazione pratica e divertente che conferma il corretto funzionamento dell'ambiente di simulazione e di analisi.

Ho realizzato questo progetto per consolidare competenze pratiche in analisi di traffico e sicurezza applicativa.

Venezia, 17/10/2025

Fabio Renna.